STAT 185 - Homework #2

MATT KRASNOW

This assignment covers multivariate probability and Principal Component Analysis through a mixture of theory and coding. You will need the file *pset2.Rdata* available on Canvas. The commands for loading the data assume this file, *pset2.Rmd*, and the Rdata file are in the same folder.

Complete the following questions and submit your solutions as a *pdf* on Gradescope by Friday September 26th at 11:59 PM.

Problem 1.

Suppose we have k linearly independent unit length vectors, $\vec{v}_1, \ldots, \vec{v}_k \in \mathbb{R}^d$ with $k < d$. Let $a_1, \ldots, a_k$ be *iid* normal random variables with mean 0 and variance ρ^2. Let $\vec{x} = a_1\vec{v_1} + \cdots + a_k\vec{v}_k + \vec{\epsilon}$ where $\vec{\epsilon} \sim \mathcal{N}(\vec{0}, \sigma^2\mathbf{I}_d)$.

a) Find the mean, covariance, and distribution of $\vec{x}$.

Answer: The mean of x can be found by the following logic. If we take the expectation of $\vec{x}$, we get $E[\vec{x}] = 0 \cdot \vec{v}_1 + \cdots + 0 \cdot \vec{v}_k + 0 = \vec{0}$
The covariance of $\vec{x}$ can be found using linearity rules.
We can express x as $x = Va + \epsilon$ where $V = v_1, \cdots v_k$ and $a = a_1, \cdots a_k$.
Solve: $Cov(x) = Cov(Va + \epsilon) = Cov(Va) + Cov(\epsilon)$
because we know that epsilon is independent of Va
Since V is a not a random variable and instead is a vector of scalars, we can express: $= VCov(a)V^T$.
The $Cov(a)$ is known to be a diagonal matrix with ρ^2 along the diagonal. Thus
$Cov(Va) = V\rho^2 I_k V^T = \rho^2 VV^T$
Thus the covariance of x is $Cov(x) = \rho^2 VV^T + \sigma^2\mathbf{I_d}$
For finding the distribution of x, we can investigate each x. We know that the as are distributed normally. This means that multiplying this value by a fixed unit vector v_i yields a vector with each value being equal to a normally distributed value times a scalar(v_{ij}). This means that each vector v_i is distributed multivariate normal $\sim MVN(0, \rho^2)$. We know that the sum of normal random variables is also normal. Thus the distribution of x is MVN $x \sim MVN(0, \rho^2 VV^T + \sigma^2\mathbf{I_d})$

b) Using the same $\vec{v}_1, \ldots, \vec{v}_k$, one thousand independent samples of $\vec{x}$ were generated as above in $\mathbb{R}^{100}$. These samples are saved in the data matrix *problem1*. Estimate k, ρ^2, and σ^2 from these data.

```
N <- nrow(problem1); d <- ncol(problem1)
S <- cov(problem1)

# decomp
eig <- eigen(S, symmetric = TRUE)
lambda <- eig$values  # eigenvalues (variances along PCs)

# use scree idea
log_drops <- -diff(log(lambda))
k_hat <- which.max(log_drops)
```

```r
# sigma^2 is average of those after the top k eigen
sigma2_hat <- mean(lambda[(k_hat + 1):d])

# rho^2 is  excess variance in the signal subspace
rho2_hat <- mean(lambda[1:k_hat]) - sigma2_hat

cat(sprintf("Estimated k: %d\n", k_hat))
```

```
## Estimated k: 28
```

```r
cat(sprintf("Estimated sigma^2: %.6f\n", sigma2_hat))
```

```
## Estimated sigma^2: 1.462814
```

```r
cat(sprintf("Estimated rho^2: %.6f\n", rho2_hat))
```

```
## Estimated rho^2: 7.627988
```

c) Suppose we wanted to estimate $\vec{v}_1, \ldots, \vec{v}_k$. What additional/alternative assumptions (if any) are needed to guarantee that we can estimate the vectors $\vec{v}_1, \ldots, \vec{v}_k$ (up to sign change) with a sufficiently large sample size?

> Answer: Essentially, we need a way to ensure that we have (somewhat) uniqueness here. We should try to find assumptions that make it so that the rotation applied by the v_is is unique except for sign. If we don't have this, we will be only be able to determine the span of the vectors and not the actual directions in which they face (the values of the v_i which is what we are actually interested in. We already think they are a basis for [in this case $\mathbb{R}^28$]). We want different rotations to actually tell us the directions of the v_i. We can get this by assuming that each of the eigenvalues is distinct from each other (Instead of all having the same ρ^2, they have τ_i^2). If all the values have multiplicity 1, we know that different rotations will give distinct vectors. Thus, if we assume this, we will be able to estimate $\vec{v}_1, \ldots, \vec{v}_k$.

Problem 2

Let $\vec{x}_1, \ldots, \vec{x}_N$ be a collection of centered data vectors. Consider the problem of projecting the data onto a k-dimensional subspace to minimize the mean squared reconstruction error.

Formally, show that the k-dimensional subspace minimizing this reconstruction error is spanned by the eigenvectors of the sample covariance matrix of X corresponding to the top k eigenvalues (i.e., the first k principal component loadings).

That is, for

$$\underset{\mathbf{W} \in \mathbb{R}^{d \times k}, \mathbf{W}^\top \mathbf{W} = \mathbf{I}}{\arg\min} \frac{1}{n} \sum_{i=1}^{n} \|x_i - \mathbf{W}\mathbf{W}^\top x_i\|^2$$

show that the optimal $\mathbf{W}$ consists of the top k eigenvectors of the empirical covariance matrix of $\mathbf{X}$.

Before I go into the formal definition, I want to explain the intuition for why this works. This works because the PC loadings corresponding to the largest eigenvalues are the most influential PC loadings; they explain the most variation in the data. Therefore, it makes sense to think that using the top k gives the smallest error; no other combination of k vectors will be able to explain as much of the variation in the data as the top k.

Proof outline: Proof by contradiction. Assume that the top k PC loadings do NOT correspond to the k-dim subspace minimizing the reconstruction error. Under this assumption, the optimal k PC loadings have at least 1 eigenvalue that is "out of order" (smaller than an eigenvalue in the top k PC). We know from PCA rules and linear algebra that the larger the eigenvalue is, the more it scales the pre-image in that direction. In otherwords, the a PC loading vector has greater influence on the image than a PC loading that has a smaller eigenvalue than it. Thus, by using a a non-top-k PC loading as one of the basis vectors, we are explaining less of the variance in the data than what is achievable via using a top-k PC loading instead. In terms of the formula, projecting x_i onto the space spanned by WW^T yields a larger distance vector if you use a non-top-k PC loading instead of a top-k PC loading. This enlarges $\sum_{i=1}^{n} \|x_i - \mathbf{W}\mathbf{W}^\top x_i\|^2$ Since we are trying to MINIMIZE $\sum_{i=1}^{n} \|x_i - \mathbf{W}\mathbf{W}^\top x_i\|^2$, the optimal $\mathbf{W}$ consists of the top-k eigenvectors of the covariance matrix of X.

Proof: Assume the minimizing W does not use the top k eigenvectors. Then at least one chosen direction corresponds to a smaller eigenvalue, while a direction with a larger eigenvalue was excluded. Since directions with larger eigenvalues explain more variance, replacing the smaller one with the larger reduces the reconstruction error

$$\sum_{i=1}^{n} \|x_i - WW^\top x_i\|^2.$$

This contradicts the assumption that W was optimal. Therefore, the optimal W must consist of the top k eigenvectors.

□

Problem 3

Suppose the top k eigenvalues of the data covariance matrix are not all distinct; that is, there is multiplicity among the largest k eigenvalues.

a) Is the k-dimensional optimal subspace unique? Prove or explain.

(a) Nonuniqueness when top eigenvalues repeat. If the largest k eigenvalues are not all distinct, the minimizing subspace need not be unique. Let the covariance have eigendecomposition with eigenpairs (λ_j, u_j) ordered nonincreasingly, and suppose some eigenvalue λ_r among the top k has multiplicity $m \geq 2$. Any k-dimensional subspace that (i) contains all eigenvectors with eigenvalues $> \lambda_r$ and (ii) chooses an arbitrary $(k - \#\{\lambda_j > \lambda_r\})$-dimensional subspace inside the λ_r–eigenspace achieves the same value of the objective. Rotations within that eigenspace leave the projection error unchanged, so the optimal subspace is not unique.

b) Give an explicit example (with calculations if possible) in which the principal components loadings corresponding to a repeated eigenvalue are not unique. Interpret the implications for the directions of the principal components.

Let

$$S = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \lambda_1 = \lambda_2 = 1,\ \lambda_3 = 0,$$

and consider $k = 1$. For any unit vector $w_\theta = (\cos\theta, \sin\theta, 0)^\top$ in the span of e_1, e_2,

$$w_\theta^\top S\, w_\theta = (\cos\theta, \sin\theta, 0) \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix} = \cos^2\theta + \sin^2\theta = 1.$$

Thus every direction w_θ is a valid first principal loading and yields the same explained variance (and the same reconstruction error). The loading is *not* unique when its eigenvalue is repeated; only the eigenspace (here, the e_1–e_2 plane) is identified. For $k = 2$, any orthonormal basis of span$\{e_1, e_2\}$ gives the same optimal 2D subspace.

Problem 4.

In this problem, we will use the Breast Cancer Wisconsin (Diagnostic) data set containing 569 observations of 30 measurements taken from images of patient tumors. These columns are saved in the data frame *cancer*. The first two columns provide i) patient id numbers and ii) labels of benign (B) or malignant (M) tumors and should be removed before running PCA.

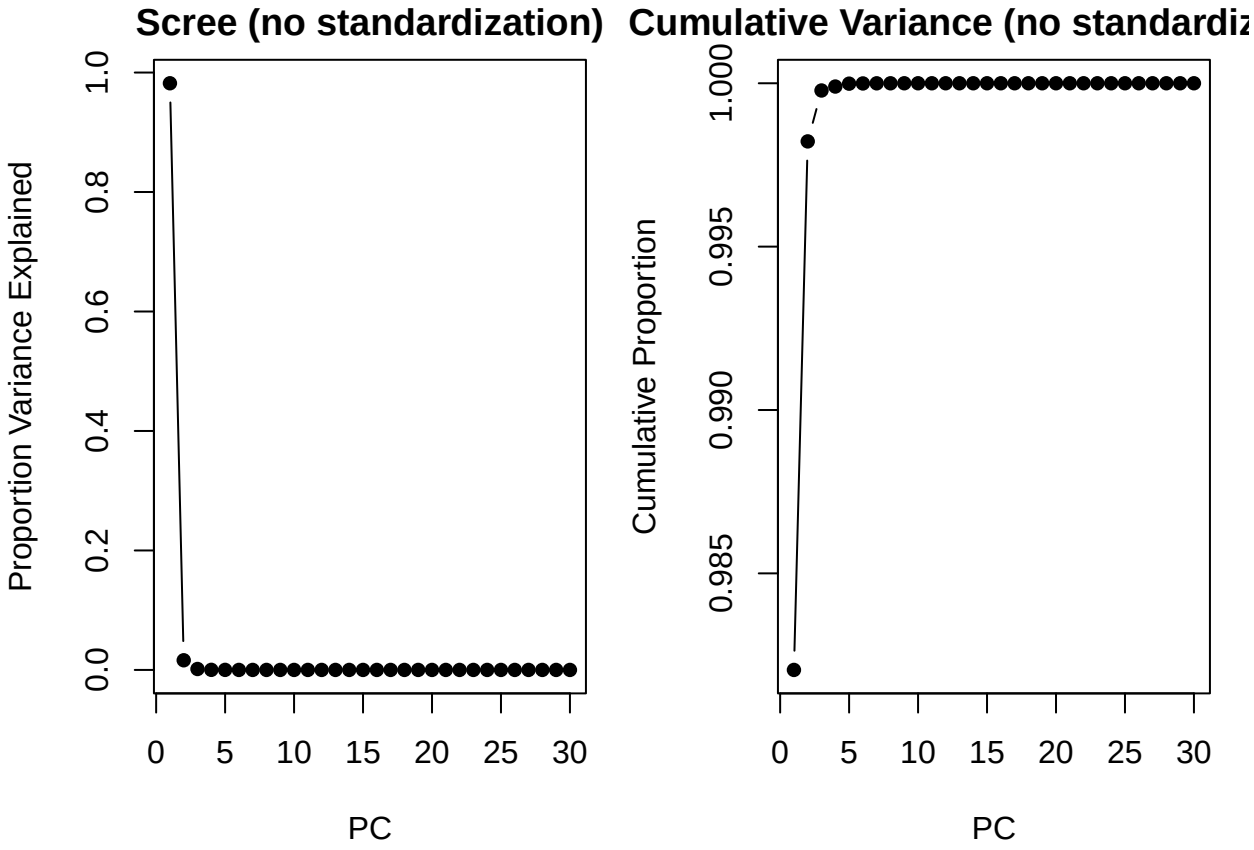
```
status <- factor(cancer[[2]], levels = c("B","M"))
X_raw  <- as.data.frame(cancer[ , -(1:2)])  # drop id + status
```

a) Apply PCA to the data without standardization. Provide scree plots and biplots summarizing PCA. Are there any outliers or imbalanced scales in the data that dominate the first few loading?

```
pca_noscale <- prcomp(X_raw, center = TRUE, scale. = FALSE)

eigvals_a <- pca_noscale$sdev^2
prop_a    <- eigvals_a / sum(eigvals_a)
cum_a     <- cumsum(prop_a)

par(mfrow = c(1,2), mar = c(4,4,2,1))
plot(prop_a, type = "b", pch = 16, xlab = "PC", ylab = "Proportion Variance Explained",
     main = "Scree (no standardization)")
plot(cum_a, type = "b", pch = 16, xlab = "PC", ylab = "Cumulative Proportion",
     main = "Cumulative Variance (no standardization)")
```

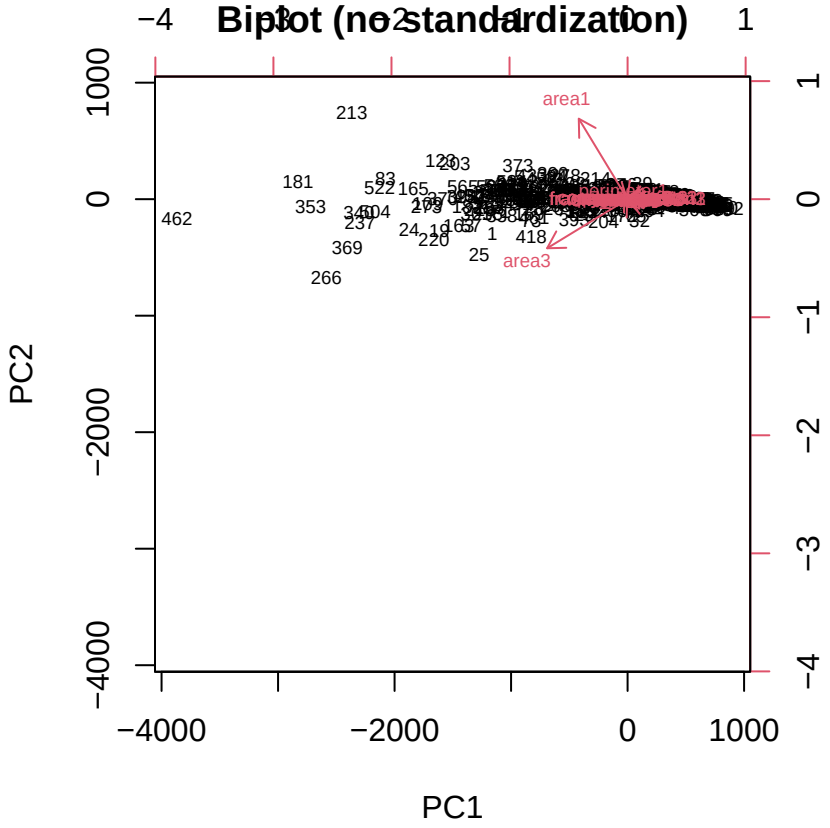



```
par(mfrow = c(1,1))

# Biplot
biplot(pca_noscale, scale = 0, cex = 0.6, main = "Biplot (no standardization)")
```

```
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
```

```
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
## Warning in arrows(0, 0, y[, 1L] * 0.8, y[, 2L] * 0.8, col = col[2L], length =
## arrow.len): zero-length arrow is of indeterminate angle and so skipped
```




```
var_sds <- sapply(X_raw, sd)


o_sds <- sort(var_sds, decreasing = TRUE)
print(round(head(o_sds, 8), 4))
```

```
##      area3      area1      area2 perimeter3 perimeter1   texture3    radius3
##   569.3570   351.9141    45.4910    33.6025    24.2990     6.1463     4.8332
##   texture1
```

```
##     4.3010
```

```
print(round(tail(o_sds, 8), 4))
```

```
## fractal_dimension3       compactness2        smoothness1          symmetry2
##             0.0181             0.0179             0.0141             0.0083
## fractal_dimension1    concave_points2        smoothness2 fractal_dimension2
##             0.0071             0.0062             0.0030             0.0026
```

```
load_a <- pca_noscale$rotation[, 1:2]
top_pc1 <- sort(abs(load_a[,1]), decreasing = TRUE)
top_pc2 <- sort(abs(load_a[,2]), decreasing = TRUE)
cat("\nTop |loading| on PC1 (no standardization):\n")
```

```
##
## Top |loading| on PC1 (no standardization):
```

```
print(head(top_pc1, 8))
```

```
##       area3       area1       area2  perimeter3  perimeter1     radius3
## 0.852063392 0.516826469 0.055727167 0.049457645 0.035076330 0.007154733
##     radius1    texture3
## 0.005086232 0.003067366
```

```
cat("\nTop |loading| on PC2 (no standardization):\n")
```

```
##
## Top |loading| on PC2 (no standardization):
```

```
print(head(top_pc2, 8))
```

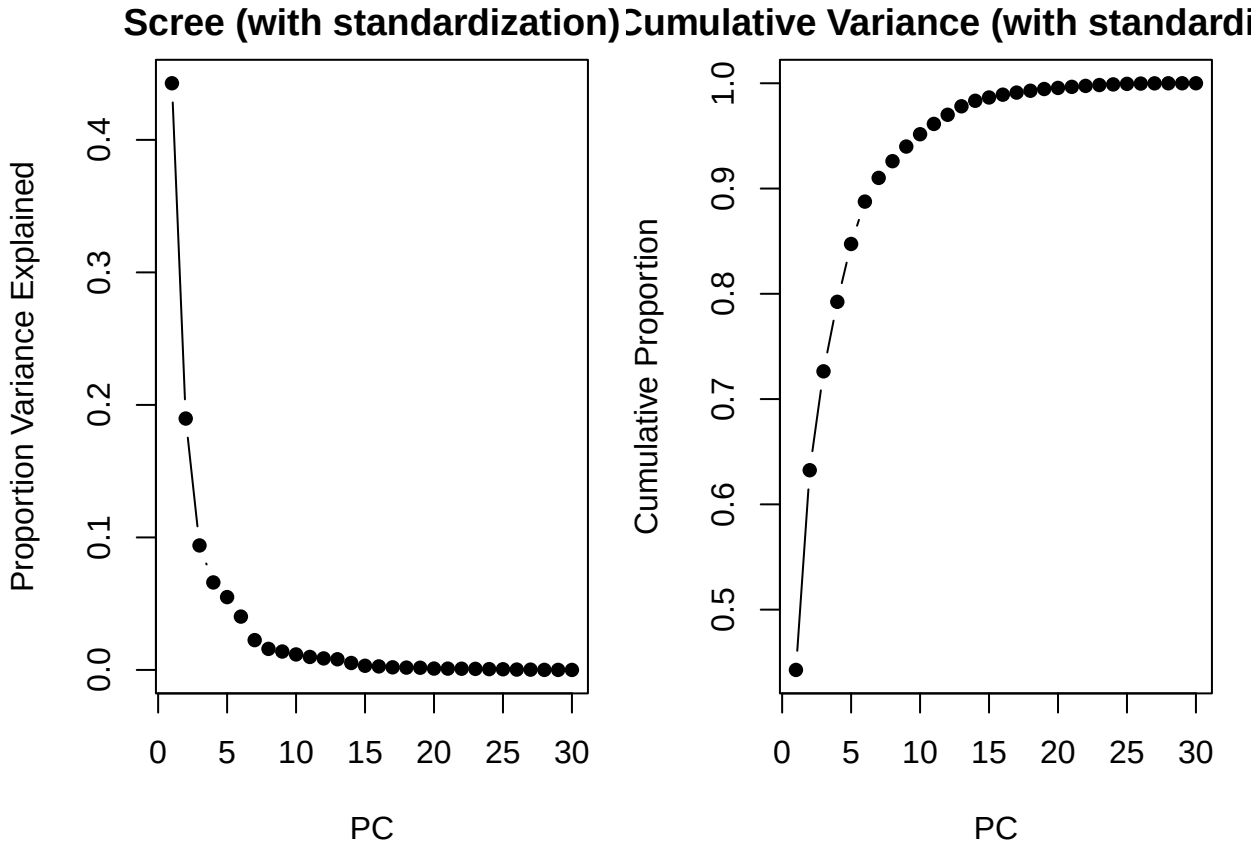
```
##        area1        area3   perimeter1     texture3      radius1        area2
## 0.8518237205 0.5197423583 0.0627480827 0.0132152605 0.0092870565 0.0075111245
##     texture1   perimeter2
## 0.0028816066 0.0008196408
```

b) Repeat the analysis in part (a) but standardize the data first.

```
pca_scale <- prcomp(X_raw, center = TRUE, scale. = TRUE)

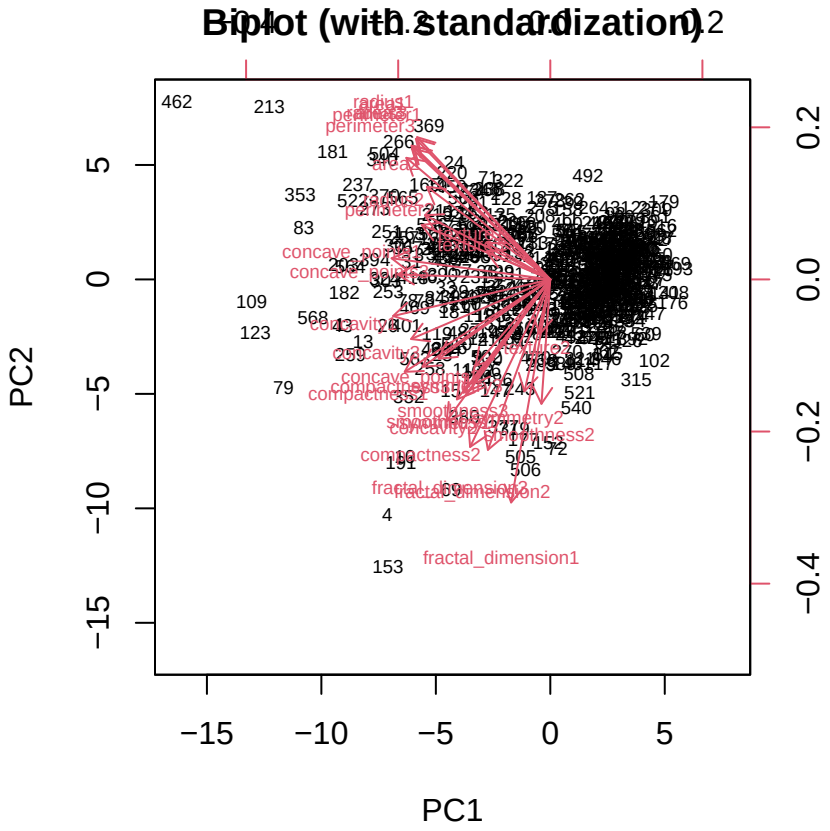
eigvals_b <- pca_scale$sdev^2
prop_b    <- eigvals_b / sum(eigvals_b)
cum_b     <- cumsum(prop_b)

par(mfrow = c(1,2), mar = c(4,4,2,1))
plot(prop_b, type = "b", pch = 16, xlab = "PC", ylab = "Proportion Variance Explained",
     main = "Scree (with standardization)")
plot(cum_b, type = "b", pch = 16, xlab = "PC", ylab = "Cumulative Proportion",
     main = "Cumulative Variance (with standardization)")
```




```
par(mfrow = c(1,1))

# Biplot
biplot(pca_scale, scale = 0, cex = 0.6, main = "Biplot (with standardization)")
```



c) For both PCA with and without standardization, plot the first two PC scores and color points by tumor status (M or B). Is separation of the classes better in one of two implementations?

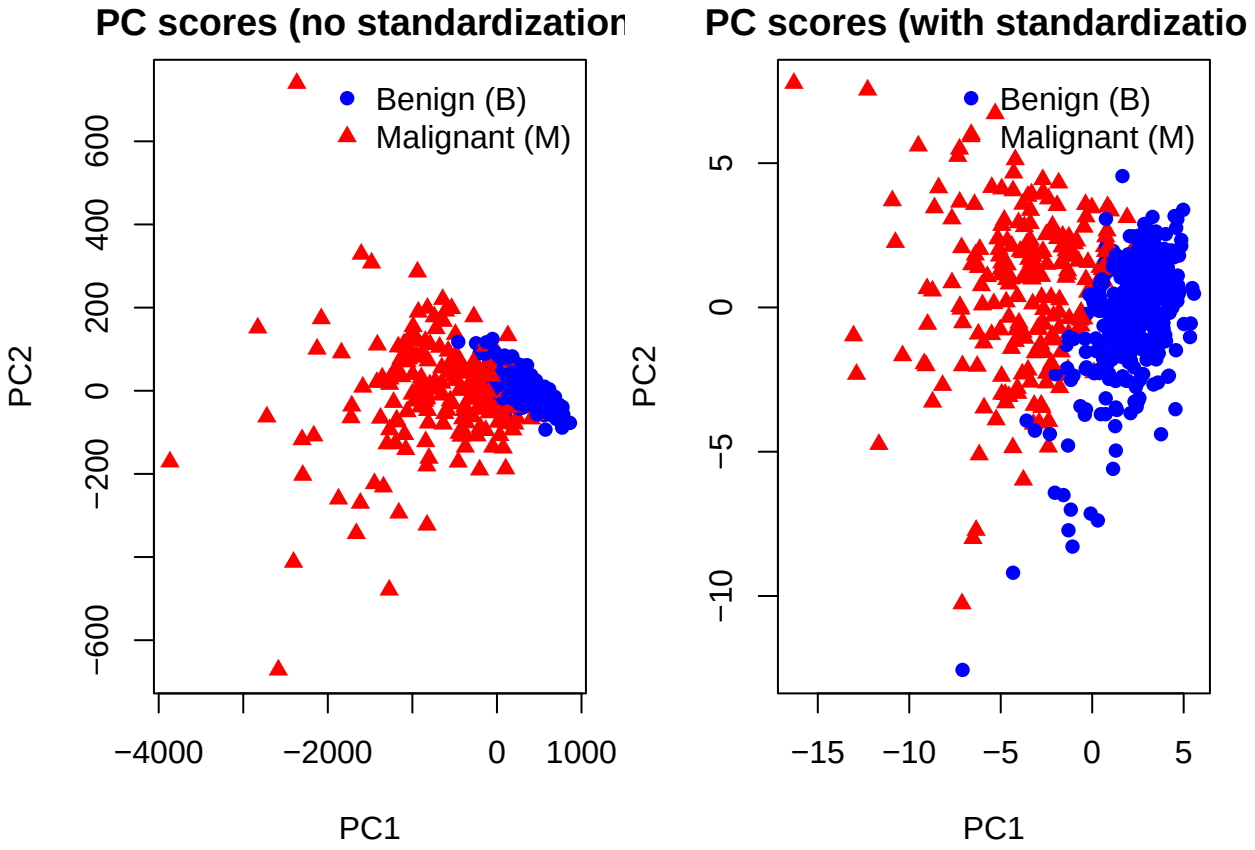
```
cols <- ifelse(status == "M", "red", "blue")
pchv <- ifelse(status == "M", 17, 16)

scores_a <- as.data.frame(pca_noscale$x[, 1:2])
scores_b <- as.data.frame(pca_scale$x[, 1:2])
names(scores_a) <- c("PC1","PC2")
names(scores_b) <- c("PC1","PC2")

par(mfrow = c(1,2), mar = c(4,4,2,1))

plot(scores_a$PC1, scores_a$PC2,
     col = cols, pch = pchv,
     xlab = "PC1", ylab = "PC2",
     main = "PC scores (no standardization)")
legend("topright", legend = c("Benign (B)", "Malignant (M)"),
       col = c("blue","red"), pch = c(16,17), bty = "n")

plot(scores_b$PC1, scores_b$PC2,
     col = cols, pch = pchv,
     xlab = "PC1", ylab = "PC2",
     main = "PC scores (with standardization)")
legend("topright", legend = c("Benign (B)", "Malignant (M)"),
       col = c("blue","red"), pch = c(16,17), bty = "n")
```




```
par(mfrow = c(1,1))
```

Standarizsation gets better separation; we can moreclearly see a 'line' which separates the two clusters. The line is pretty clearly a function of PC1 and PC2 and easier to represent. We would prefer the standardization because it leads to a more interpretable and successful separation of the groups.

Problem 5.

In this problem, we investigate the ability of PCA to compress image data using the Olivetti faces data set. Each image has been flattened into a data vector and saved in the data matrix *faces* with one image per row. The helper function *plot_image* allows you to visualize the associated image and compressed versions generated using PCA.

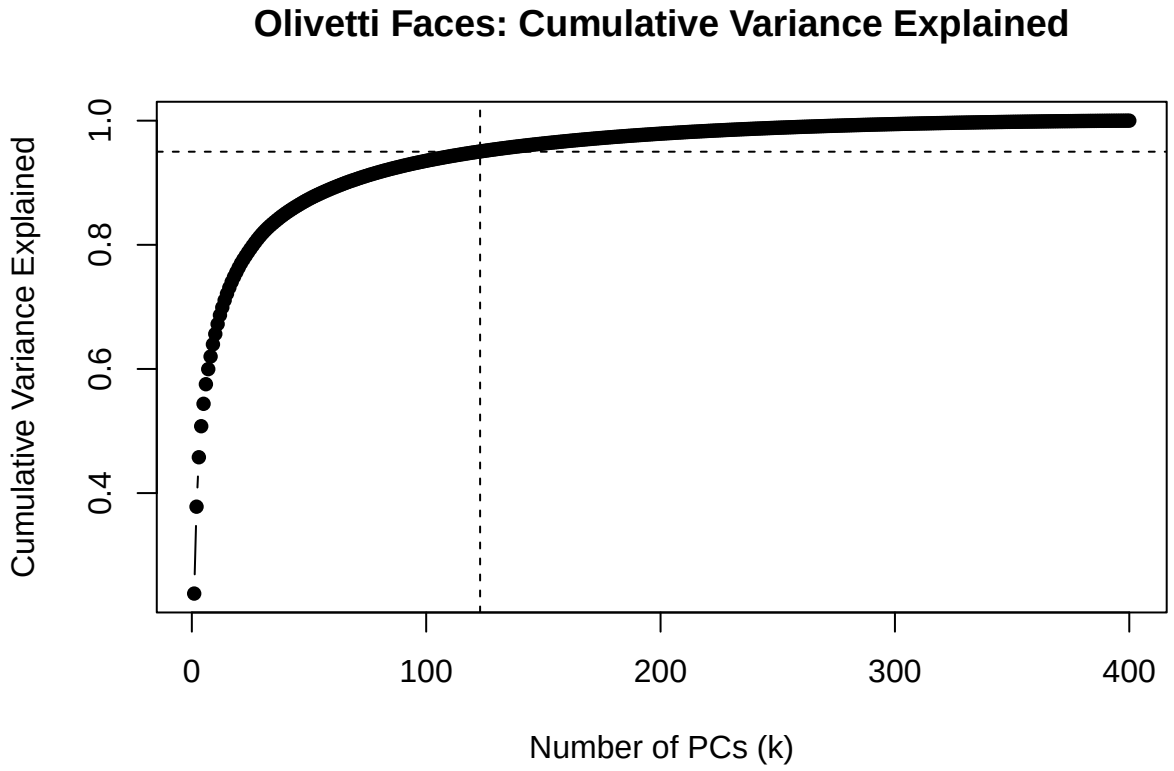
```
plot_image(faces[1,])
```



a) Apply PCA to the *faces* data matrix without standardization. Provide a plot showing the total % of variability explained by the first k components. How many components are needed to account for 95% of the variability in the data?

```r
stopifnot(exists("faces"))
pca_faces <- prcomp(faces, center = TRUE, scale. = FALSE)
eigvals_f <- pca_faces$sdev^2
prop_f    <- eigvals_f / sum(eigvals_f)
cum_f     <- cumsum(prop_f)
k95 <- which(cum_f >= 0.95)[1]

plot(cum_f, type = "b", pch = 16, xlab = "Number of PCs (k)", ylab = "Cumulative Variance Explained",
     main = "Olivetti Faces: Cumulative Variance Explained")
abline(h = 0.95, lty = 2)
abline(v = k95, lty = 2)
```

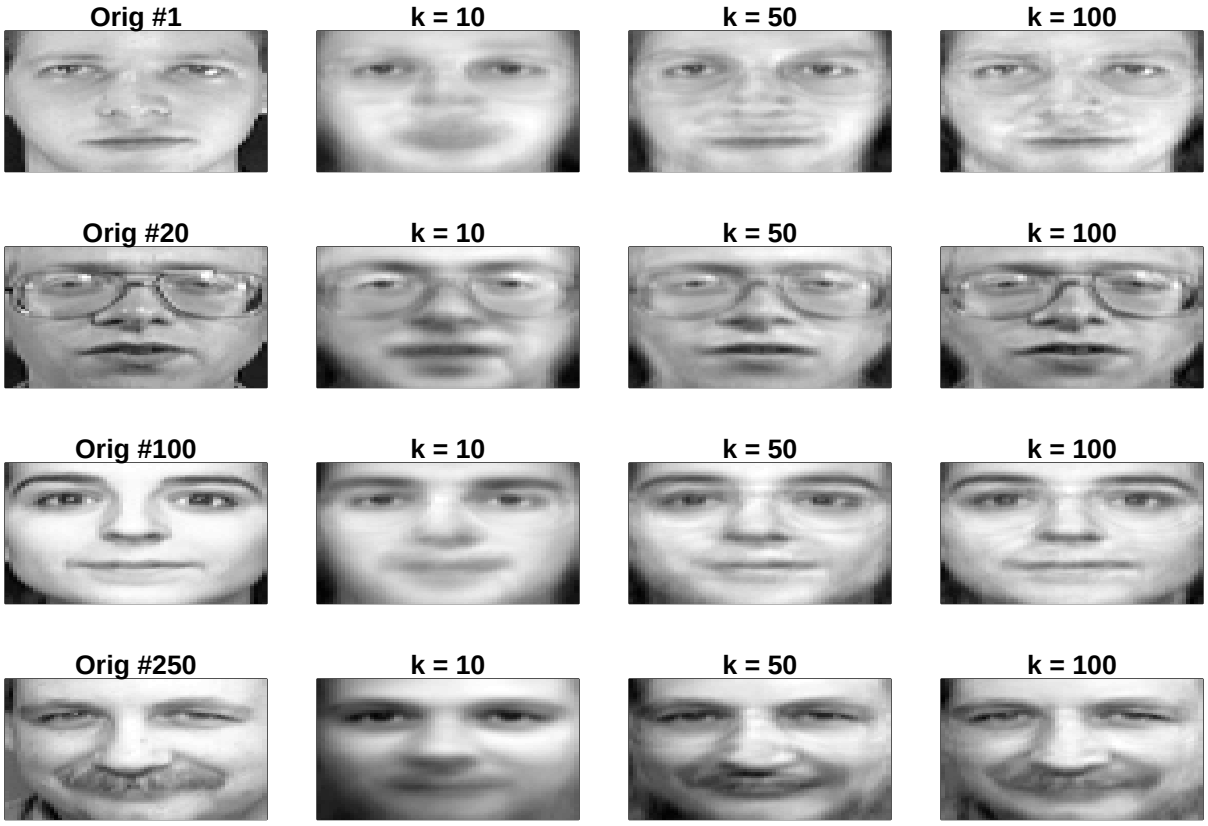


Without standardization, the number of principal components needed is 123

```r
reconstruct_with_k <- function(X, pca_obj, k) {
  Xm <- sweep(X, 2, pca_obj$center, "-")
  scores_k <- Xm %*% pca_obj$rotation[, 1:k, drop = FALSE]
  Xhat <- scores_k %*% t(pca_obj$rotation[, 1:k, drop = FALSE])
  sweep(Xhat, 2, pca_obj$center, "+")
}

idx <- c(1, 20, 100, 250)
ks  <- c(10, 50, 100)
Xhat10  <- reconstruct_with_k(faces[idx, , drop = FALSE], pca_faces, ks[1])
Xhat50  <- reconstruct_with_k(faces[idx, , drop = FALSE], pca_faces, ks[2])
Xhat100 <- reconstruct_with_k(faces[idx, , drop = FALSE], pca_faces, ks[3])
```

```r
op <- par(mfrow = c(4,4), mar = c(1,1,2,1))
for (i in seq_along(idx)) {
  plot_image(faces[idx[i], ])
  title(main = sprintf("Orig #%d", idx[i]), line = 0.2)
  plot_image(Xhat10[i, ])
  title(main = "k = 10", line = 0.2)
  plot_image(Xhat50[i, ])
  title(main = "k = 50", line = 0.2)
  plot_image(Xhat100[i, ])
  title(main = "k = 100", line = 0.2)
}
```




```r
par(op)
```

b) Write a function which approximates each image using the first k PC components. Show the first, 20th, 100th, 250th images along with the resconstructions using 10, 50 and 100 principal components.

(5b) The grid shows each original image (rows: 1, 20, 100, 250) alongside PCA reconstructions using k $\in 10, 50, 100$ components. Visual quality improves as k increases, reflecting higher retained variance.

Problem 6

Suppose the original samples variances of each column of a data matrix, $\mathbf{X} \in \mathbb{R}^{N \times d}$ are $\sigma_1^2, \ldots, \sigma_d^2$. Let $\tilde{\mathbf{X}}$ be the data matrix after standardization is applied to $\mathbf{X}$, i.e. each column is centered and rescaled to have sample variance one.

a) Express $\tilde{\mathbf{X}}$ as a product of the centering matrix, the original data matrix, and a diagonal matrix depending on the original variances of the columns of $\mathbf{X}$.

Answer: To center the data, we can do HX where H is the centering matrix. To standardize the data so that it has sample variance = 1, we can divide each term in each column by the corrosponding σ value. Define $C = \mathbf{I_d} * [\sigma_1, \cdots, \sigma_d]$. Thus, $\tilde{X} = HXC^{-1}$

b) Suppose PCA is applied to $\tilde{\mathbf{X}}$. Provide a formula for the reconstruction of $\mathbf{X}$ *unstandardizing* the first k principal components of $\tilde{\mathbf{X}}$.

Answer: This is equivilent to multiplying by the sigmas again so that we have the unstandardze variances reincorporated. Then we just add the means of each column back to the data. Let's define the following notation to show this in a formula:
$\hat{\tilde{X}}_k$ is the PCA-applied matrix of data with k as the selected number of PC loadings kept.
μ is a column vector of the means of each of the columns in X
Thus to apply the process I described:

$$\hat{X_k} = \tilde{X}C + \vec{1}\mu^T$$

UL in Action: PCA and Genome-Wide Association Studies

Problem 7.

Genome-Wide Association Studies (GWAS) are an observational study design used to assess the association of genetic variants, usually single-nucleotide polymorphism (SNPs), with specific traits, such as major diseases. GWAS have been used extensively in agriculture and human health.

If you are interested in additional details on PCA in this context including a deep dive on random matrix theory, check out this wonderful pair of lectures from the Broad Institute.

First, a brief introduction. DNA is a long chain of nucleotides (G)uanine, (A)denine, (C)ytosine and (T)hymine, meaning any gene or genome can be represented as a long string of G, A, C, and Ts. To use PCA, we first need to convert "textual" genomics data to numerical data.

At a specific location in the genome, suppose most individuals have a G nucleotide. However, there may be a small subset of the population that has an A nucleotide instead. In this case, we say there is a SNP at this location in the genome with two possible alleles, G and A. Humans have two chromosomes, so we could check this specific location in both chromosomes. An individual could have a G nucleotide at this location in both chromosomes (GG), an A nucleotide on one chromosome and a G on the other (GA or AG), or an A nucleotide in both locations (AA). Thus, we could summarize the genetic variations at the specific location as GG, AG, GA, or AA.

In this case, GG is the most common variant across the population, so we will use it as a reference. To convert the nucleotide pairs to numerical data, we simply count the number of alleles which differ from the reference GG. Thus, GG gets converted to a 0, AG and GA get converted to 1, and AA gets converted to a 2. Repeating this process over every SNP and individual allows us to represent the genome of the individuals in our study as long vectors of zeros, ones, and twos, e.g. $\vec{x}_i = \{0, 1, 2\}^d$ for $i = 1, \ldots, N$.

In a GWAS we have corresponding set of observed traits, y_i for $i = 1, \ldots, N$ measuring a trait of interest for each individual. We could then use the paired observations $(\vec{x}_i, y_i)$ to learn a model

$$y = f(x)$$

allowing us to predict the trait as a function of the genetic variation in our study population. Such an approach is fraught with limitations and challenges both technical and ethical. In this case, we're most interested in a specific problem which can arise as a function of population structure. Here is a simple example of the problem.

Suppose our trait of interest is height and our study population has samples from individuals in two or more geographically separated regions of the world. It is reasonable to expect that the genomes in each region will have evolved differences (SNPs) in one or more nucleotides over time, and most of these differences will have no effect on height. However, if there are natural differences in height between these regions due to other confounding factors, naively using our genomics data may result in a model f, which incorrectly associates variation at certain SNPs with increases or decreases in height. To address this confounding, we need a way of accounting for the natural variation in the genome between these two subpopulations. One approach would be to include a variable in our model f indicating the populations from which the sample was drawn. In this problem, we are going to explore an alternative approach where PCA is used as a preprocessing step in GWAS to allow for population stratification as a method to prevent confounding.

Below, we have loaded data from 279 individuals in the HapMap database into the matrix *geno.*

```
genofile <- snpgdsOpen(snpgdsExampleFileName())
# reproducing SNP PCA directly via SVD
snpset <- snpgdsLDpruning(genofile,
                          ld.threshold=0.3,
                          maf=0.001,
                          missing.rate=0.001,
                          verbose = FALSE)
snpset.id <- unlist(unname(snpset))
geno <- snpgdsGetGeno(genofile, snp.id = snpset.id)
```

```
## Genotype matrix: 279 samples X 6017 SNPs
```

```
pop_code <- read.gdsn(index.gdsn(genofile, "sample.annot/pop.group"))
```

Here each row corresponds to one of the 279 people from the HapMap database. Each column corresponds to one of the 6017 SNPs. The first three rows and five columns of *geno* are shown below.

2	1	0	2	1
1	1	0	2	2
2	1	1	2	2

Note, each genome also has an associated location where the individual was living when their DNA was sequenced. These locations are stored in the variable *pop_code* where the entries correspond to the following populations/locations.

Code	Population
YRI	Yoruba in Ibadan, Nigeria
CEU	Utah residents with Northern and Western European ancestry
HCB	Han Chinese in Beijing, China
JPT	Japanese in Tokyo, Japan

In this problem, we will perform PCA on the *geno* data and see that clustering of these subpopulations emerges when one looks at the PC scores. We first follow the centering and normalizing process from *Principal components analysis corrects for stratification in genome-wide association studies* by Price et al.

a) Given data matrix $X \in \mathbb{R}^{N\times d}$, let $\bar{x} \in \mathbb{R}^d$ be the sample mean of the data. We may view $\bar{x}$ as the column mean of X. We rescale each entry of X:

$$X_{ij} \mapsto \frac{X_{ij} - \bar{x}_j}{\sqrt{p_j(1-p_j)}} \qquad \text{for} \qquad 1 \le j \le d, 1 \le i \le N$$

where

$$p_j = \frac{1 + N\bar{x}_j}{2 + 2N} = \frac{1/N + \bar{x}_j}{2/N + 2}$$

is an estimate of the allele frequency of SNP j. Apply this transformation to the data matrix *geno*. Show the first three rows and five columns of the centered and normalized matrix that results from this transformation.

```
X <- as.matrix(geno)
N <- nrow(X); d <- ncol(X)

xbar <- colMeans(X)
p_j  <- (1 + N * xbar) / (2 + 2 * N)

den  <- sqrt(p_j * (1 - p_j))
eps  <- 1e-8
den[den < eps] <- eps

X_tilde <- sweep(X, 2, xbar, "-")
X_tilde <- sweep(X_tilde, 2, den, "/")
knitr::kable(round(X_tilde[1:3, 1:5, drop = FALSE], 4), "simple")
```

1.3118	1.464	-0.4440	0.7289	-1.4640
-0.8667	1.464	-0.4440	0.7289	1.0115
1.3118	1.464	4.1443	0.7289	1.0115

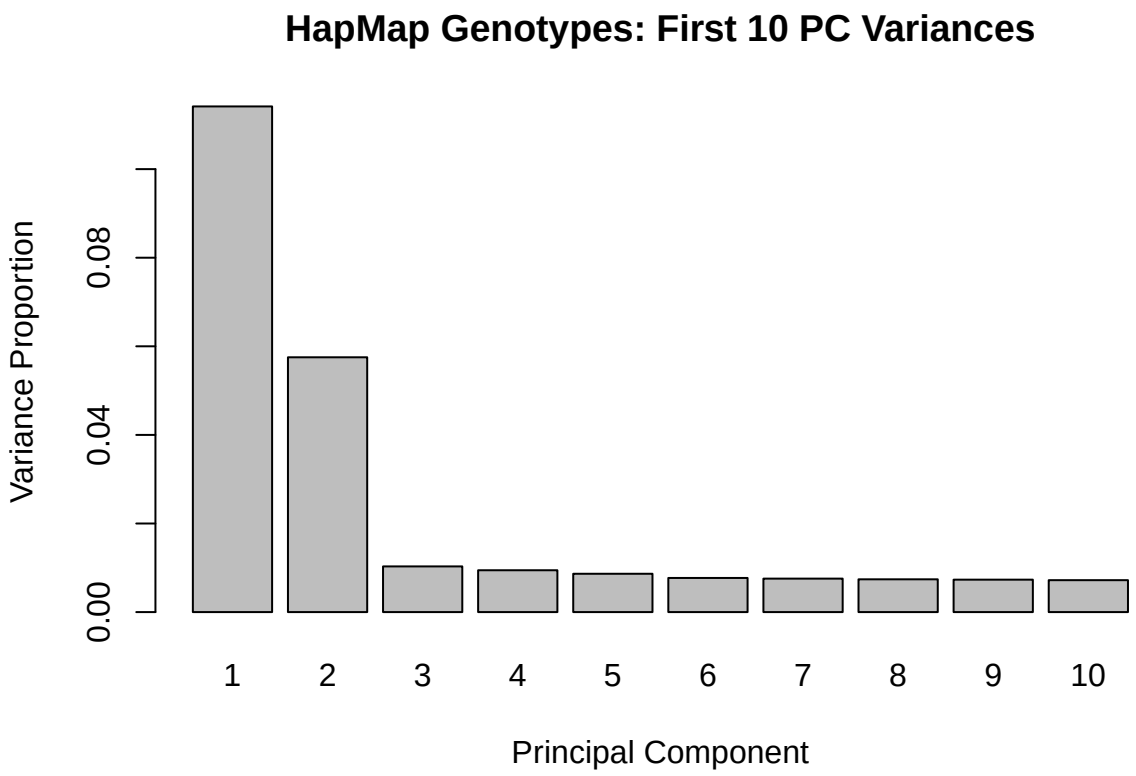
The table shows the first three rows and first five columns of the centered-and-normalized genotype matrix $\tilde{X}$ using $X_{ij} \mapsto (X_{ij} - \bar{x}_j)/\sqrt{p_j(1-p_j)}$ with $p_j = \dfrac{1 + N\bar{x}_j}{2 + 2N}$.

b) Apply PCA to the centered and normalized data matrix. Plot the first 10 principal component variances. Do these results suggest there is low dimensional structure in the *geno* data? Explain.

```
pca_g <- prcomp(X_tilde, center = FALSE, scale. = FALSE)

eigvals_g <- pca_g$sdev^2
prop_g    <- eigvals_g / sum(eigvals_g)

k_show <- 10
barplot(prop_g[1:k_show],
        xlab = "Principal Component",
        ylab = "Variance Proportion",
        main = "HapMap Genotypes: First 10 PC Variances",
        names.arg = 1:k_show)
```



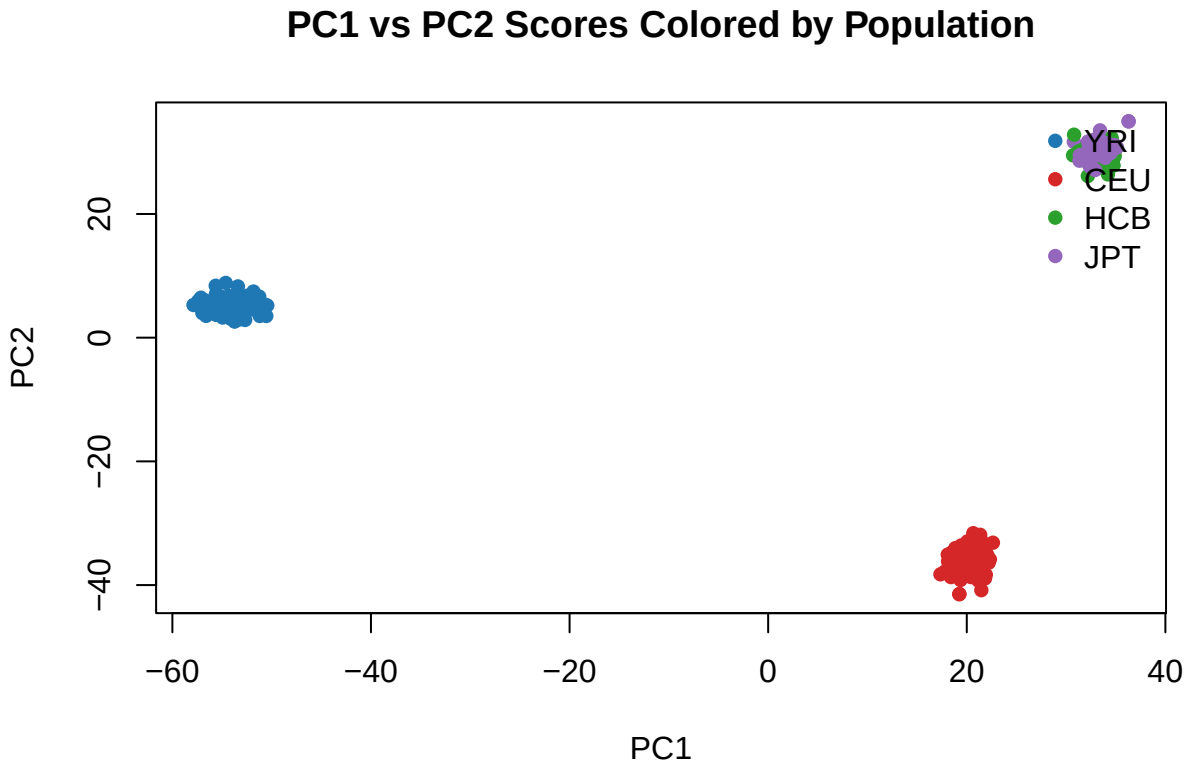

```
p2   <- sum(prop_g[1:2])
p10  <- sum(prop_g[1:10])
```

The leading components don't capture much of the share of variability (e.g., PC1–2 together explain about 17.2%; PC1–10 explain about 23.7%). This concentration DOES NOT indicates meaningful low-dimensional structure in the genotype data beyond pure noise.

c) In GWAS the first few PC scores are typically included in the model f to account for the natural variation the occurs across the subpopulations. Plot first two principal component scores. Color the points by location from which the sample was taken using the *pop_code* variable. Discuss the clustering in the PC scores based on location. In particular, could PC scores be used as a proxy for the locations from which are data was sampled? In the article *Genes mirror geography within Europe* by Novembre et at, we can see a more detailed implementation of this idea. Unfortunately, it requires access to the POPRES database, which is access restricted.

```
scores <- as.data.frame(pca_g$x[, 1:2, drop = FALSE])
colnames(scores) <- c("PC1","PC2")
grp <- factor(pop_code, levels = c("YRI","CEU","HCB","JPT"))
cols <- c("YRI"="#1f77b4","CEU"="#d62728","HCB"="#2ca02c","JPT"="#9467bd")[grp]

plot(scores$PC1, scores$PC2,
     col = cols, pch = 16,
     xlab = "PC1", ylab = "PC2",
     main = "PC1 vs PC2 Scores Colored by Population")
legend("topright",
       legend = levels(grp),
       col = c("#1f77b4","#d62728","#2ca02c","#9467bd"),
       pch = 16, bty = "n")
```



The PC1–PC2 score plot shows clear clustering aligned with population labels (YRI, CEU, HCB, JPT). Because these clusters separate well in the first two PCs, the leading PC scores can serve as effective proxies for population structure in GWAS models, helping control confounding from stratification when included as covariates.